GYCS Crew Hub — Architecture Plan

> **Step 1 of the Working Method.** This document defines the file tree, database schema, RLS policies, route map, feature flags, and open decisions before any code is written.

1. File Tree

```
gycs-crew-hub/
├── .env.local.example                # Secret placeholders
├── .eslintrc.json
├── next.config.ts
├── tailwind.config.ts
├── tsconfig.json
├── package.json
├── README.md
│
├── supabase/
│   ├── config.toml
│   ├── seed.sql                      # Dev seed data
│   └── migrations/
│       ├── 00001_identity.sql        # orgs, profiles, roles, consent
│       ├── 00002_pathways.sql        # assessments, pathway_versions,
requirements, milestones
│       ├── 00003_learning.sql        # programmes, cohorts, lessons, assignments,
attendance
│       ├── 00004_evidence.sql        # documents, credentials, reviews,
audit_events
│       ├── 00005_career.sql          # cv_versions, employers, opportunities,
applications, interviews
│       ├── 00006_outcomes.sql        # offers, verified_starts, retention_checks,
withdrawal_reasons
│       ├── 00007_rls_policies.sql    # All RLS policies in one reviewable file
│       ├── 00008_functions.sql       # DB functions (transition guards, audit
triggers)
│       └── 00009_feature_flags.sql   # Feature-flag table + seed
│
├── public/
│   ├── favicon.ico
│   └── images/
│
├── src/
│   ├── middleware.ts                 # Auth guard, role routing
│   │
│   ├── lib/
│   │   ├── supabase/
│   │   │   ├── client.ts            # Browser client
│   │   │   ├── server.ts            # Server-side client (cookies)
│   │   │   ├── admin.ts             # Service-role client (background jobs only)
│   │   │   ├── types.ts             # Generated DB types
│   │   │   └── rls-test-helpers.ts  # Test utilities for RLS
```

```
│   │   ├── constants.ts              # Journey states, role enums, readiness
dimensions
│   │   ├── feature-flags.ts          # Runtime flag checks
│   │   ├── guardrails.ts             # Copy-validation helpers (no guarantee
language)
│   │   ├── ai/
│   │   │   ├── cv-assist.ts          # Server-only CV wording aid
│   │   │   ├── lesson-search.ts      # Semantic lesson search
│   │   │   └── summarise.ts          # Session summarisation
│   │   └── integrations/
│   │       ├── ghl-sync.ts           # GoHighLevel downstream sync
│   │       ├── zoom.ts               # Zoom Meeting SDK helpers
│   │       └── cloudflare-stream.ts  # Signed playback (feature-flagged)
│   │
│   ├── app/
│   │   ├── layout.tsx                # Root layout, font, metadata
│   │   ├── page.tsx                  # Public landing / marketing
│   │   ├── globals.css
│   │   │
│   │   ├── (auth)/
│   │   │   ├── login/page.tsx
│   │   │   ├── register/page.tsx
│   │   │   ├── forgot-password/page.tsx
│   │   │   └── callback/route.ts     # Supabase OAuth callback
│   │   │
│   │   ├── (member)/                 # Authenticated member layout group
│   │   │   ├── layout.tsx            # Sidebar/bottom-nav, role guard
│   │   │   ├── today/page.tsx
│   │   │   ├── intake/
│   │   │   │   ├── page.tsx          # "Find my route" wizard
│   │   │   │   └── result/page.tsx   # Assessment outcome
│   │   │   ├── pathway/
│   │   │   │   ├── page.tsx          # Milestone timeline
│   │   │   │   └── [milestoneId]/page.tsx
│   │   │   ├── learn/
│   │   │   │   ├── page.tsx          # Programme/lesson list
│   │   │   │   ├── [lessonId]/page.tsx
│   │   │   │   └── assignments/
│   │   │   │       ├── page.tsx
│   │   │   │       └── [assignmentId]/page.tsx
│   │   │   ├── live/
│   │   │   │   ├── page.tsx          # Session calendar
│   │   │   │   └── [sessionId]/page.tsx
│   │   │   ├── documents/
│   │   │   │   ├── page.tsx          # Upload / status list
│   │   │   │   └── [documentId]/page.tsx
```

```
│   │   │   ├── cv/
│   │   │   │   ├── page.tsx          # CV Studio editor
│   │   │   │   ├── preview/page.tsx  # PDF preview
│   │   │   │   └── history/page.tsx  # Version history
│   │   │   ├── explore/
│   │   │   │   ├── page.tsx          # Destination resources
│   │   │   │   └── map/page.tsx      # MapLibre directory
│   │   │   ├── opportunities/
│   │   │   │   ├── page.tsx          # Approved partner listings
│   │   │   │   └── [opportunityId]/page.tsx
│   │   │   ├── applications/
│   │   │   │   ├── page.tsx          # Saved / submitted / tracking
│   │   │   │   └── [applicationId]/page.tsx
│   │   │   ├── support/
│   │   │   │   └── page.tsx          # Coach requests, feedback
│   │   │   └── settings/
│   │   │       ├── page.tsx          # Profile, consent, export, delete
│   │   │       └── consent/page.tsx  # POPIA consent management
│   │   │
│   │   ├── (coach)/                  # Authenticated coach layout group
│   │   │   ├── layout.tsx
│   │   │   ├── console/page.tsx      # Intervention-first dashboard
│   │   │   ├── members/
│   │   │   │   ├── page.tsx          # Assigned members
│   │   │   │   └── [memberId]/page.tsx # Deep member view
│   │   │   ├── reviews/page.tsx      # Pending evidence reviews
│   │   │   ├── cohorts/
│   │   │   │   ├── page.tsx
│   │   │   │   └── [cohortId]/page.tsx
│   │   │   ├── outcomes/page.tsx     # Outcome recording
│   │   │   └── settings/page.tsx
│   │   │
│   │   └── api/
│   │       ├── webhooks/
│   │       │   ├── supabase/route.ts # DB webhook handler
│   │       │   └── zoom/route.ts     # Zoom event webhook
│   │       ├── cron/
│   │       │   ├── document-expiry/route.ts
│   │       │   ├── reminders/route.ts
│   │       │   └── replay-process/route.ts
│   │       ├── cv/
│   │       │   ├── generate-pdf/route.ts
│   │       │   └── ai-assist/route.ts   # Feature-flagged
│   │       ├── documents/
│   │       │   └── signed-url/route.ts
│   │       └── ghl-sync/route.ts
```

```
│   │
│   ├── components/
│   │   ├── ui/                        # Primitives (Button, Card, Input, etc.)
│   │   ├── layout/
│   │   │   ├── MemberNav.tsx
│   │   │   ├── CoachNav.tsx
│   │   │   └── MobileBottomNav.tsx
│   │   ├── intake/
│   │   │   ├── IntakeWizard.tsx
│   │   │   ├── IntakeStep.tsx
│   │   │   └── IntakeResult.tsx
│   │   ├── pathway/
│   │   │   ├── MilestoneTimeline.tsx
│   │   │   └── JourneyStateIndicator.tsx
│   │   ├── documents/
│   │   │   ├── DocumentUploader.tsx
│   │   │   ├── DocumentCard.tsx
│   │   │   └── ExpiryBadge.tsx
│   │   ├── cv/
│   │   │   ├── CVEditor.tsx
│   │   │   ├── CVSection.tsx
│   │   │   ├── VerificationBadge.tsx
│   │   │   └── CVPDFPreview.tsx
│   │   ├── learn/
│   │   │   ├── LessonCard.tsx
│   │   │   ├── AssignmentSubmission.tsx
│   │   │   └── QuizRunner.tsx
│   │   ├── live/
│   │   │   ├── SessionCalendar.tsx
│   │   │   └── JoinButton.tsx
│   │   ├── explore/
│   │   │   ├── MapDirectory.tsx
│   │   │   └── PinCard.tsx
│   │   ├── opportunities/
│   │   │   ├── OpportunityCard.tsx
│   │   │   └── ApplicationTracker.tsx
│   │   ├── coach/
│   │   │   ├── InterventionQueue.tsx
│   │   │   ├── MemberCaseView.tsx
│   │   │   ├── ReadinessMatrix.tsx
│   │   │   ├── OutcomeRecorder.tsx
│   │   │   └── DiagnosticFlag.tsx
│   │   ├── shared/
│   │   │   ├── FeatureGate.tsx        # Wraps feature-flagged UI
│   │   │   ├── ConsentBanner.tsx
│   │   │   ├── AuditTrail.tsx
```

```
│   │   │   └── GuardrailWarning.tsx   # Surfaces TODO / professional-confirmation
items
│   │   └── icons/
│   │
│   ├── hooks/
│   │   ├── useUser.ts
│   │   ├── useRole.ts
│   │   ├── useFeatureFlag.ts
│   │   ├── useJourneyState.ts
│   │   └── useRealtimeUpdates.ts
│   │
│   └── types/
│       ├── database.ts               # Mirrors generated Supabase types
│       ├── intake.ts
│       ├── pathway.ts
│       ├── cv.ts
│       └── enums.ts
│
├── scripts/
│   ├── generate-types.sh             # supabase gen types typescript
│   └── rls-tests.sql                 # pgTAP / raw SQL RLS test suite
│
└── docs/
    ├── ARCHITECTURE.md
    ├── GUARDRAILS.md                  # Copy of the non-negotiable rules
    ├── DATA-MODEL.md
    ├── OPEN-DECISIONS.md
    └── FEATURE-FLAGS.md
```

2. Supabase Schema (SQL Migrations)

2.1 Migration 00001 — Identity

```
sql
-- supabase/migrations/00001_identity.sql

-- Enum types
CREATE TYPE public.user_role AS ENUM ('member', 'coach', 'admin');
CREATE TYPE public.consent_type AS ENUM (
  'programme_participation',
  'marketing_email',
  'marketing_sms',
  'marketing_whatsapp',
  'recording_consent',
  'data_export',
  'ai_cv_assist'
);
CREATE TYPE public.consent_status AS ENUM ('granted', 'withdrawn');

-- Organisations (future: multiple coaching orgs)
CREATE TABLE public.organisations (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name          text NOT NULL,
  slug          text UNIQUE NOT NULL,
  created_at    timestamptz NOT NULL DEFAULT now(),
  updated_at    timestamptz NOT NULL DEFAULT now()
);

-- User profiles (extends auth.users)
CREATE TABLE public.profiles (
  id              uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  organisation_id uuid REFERENCES public.organisations(id),
  role            public.user_role NOT NULL DEFAULT 'member',
  display_name    text NOT NULL,
  email           text NOT NULL,
  phone           text,
  country_code    text DEFAULT 'ZA',
  avatar_url      text,
  timezone        text DEFAULT 'Africa/Johannesburg',
  is_active       boolean NOT NULL DEFAULT true,
  onboarded_at    timestamptz,
  created_at      timestamptz NOT NULL DEFAULT now(),
  updated_at      timestamptz NOT NULL DEFAULT now()
);
```

```
-- Coach-to-member assignments
CREATE TABLE public.coach_assignments (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  coach_id    uuid NOT NULL REFERENCES public.profiles(id),
  member_id   uuid NOT NULL REFERENCES public.profiles(id),
  assigned_at timestamptz NOT NULL DEFAULT now(),
  ended_at    timestamptz,
  notes       text,
  UNIQUE(coach_id, member_id, assigned_at)
);

-- POPIA consent records (separate from participation)
CREATE TABLE public.consent_records (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id        uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  consent_type   public.consent_type NOT NULL,
  status         public.consent_status NOT NULL,
  granted_at     timestamptz,
  withdrawn_at   timestamptz,
  ip_address     inet,
  user_agent     text,
  created_at     timestamptz NOT NULL DEFAULT now()
);

-- Suppression list for marketing (POPIA)
CREATE TABLE public.marketing_suppressions (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  channel     text NOT NULL CHECK (channel IN ('email', 'sms', 'whatsapp')),
  suppressed  boolean NOT NULL DEFAULT true,
  updated_at  timestamptz NOT NULL DEFAULT now(),
  UNIQUE(user_id, channel)
);

-- Access log for audit
CREATE TABLE public.access_logs (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid NOT NULL,
  resource    text NOT NULL,
  action      text NOT NULL,
  metadata    jsonb DEFAULT '{}',
  ip_address  inet,
  created_at  timestamptz NOT NULL DEFAULT now()
);

-- updated_at trigger function
```

```
CREATE OR REPLACE FUNCTION public.handle_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER set_profiles_updated_at
  BEFORE UPDATE ON public.profiles
  FOR EACH ROW EXECUTE FUNCTION public.handle_updated_at();

CREATE TRIGGER set_organisations_updated_at
  BEFORE UPDATE ON public.organisations
  FOR EACH ROW EXECUTE FUNCTION public.handle_updated_at();

-- Auto-create profile on auth signup
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.profiles (id, email, display_name, role)
  VALUES (
    NEW.id,
    NEW.email,
    COALESCE(NEW.raw_user_meta_data->>'display_name', split_part(NEW.email, '@',
1)),
    'member'
  );
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

2.2 Migration 00002 — Pathways

```
sql
-- supabase/migrations/00002_pathways.sql

CREATE TYPE public.journey_state AS ENUM (
  'exploring',
  'pathway_agreed',
  'preparing',
  'training_complete',
  'profile_ready',
  'search_active',
  'interviewing',
  'offer',
  'employment_started',
  'first_season_supported',
  -- Non-linear states
  'paused',
  'needs_review',
  'alternative_route'
);

CREATE TYPE public.department AS ENUM ('deck', 'interior', 'engineering',
'galley');

CREATE TYPE public.requirement_classification AS ENUM (
  'required_for_context',
  'employer_preference',
  'useful_development',
  'defer',
  'requires_professional_confirmation'
);

-- Intake assessments
CREATE TABLE public.assessments (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  status            text NOT NULL DEFAULT 'in_progress'
                    CHECK (status IN ('in_progress', 'completed', 'reviewed')),
  -- Structured responses (no PII beyond what member enters)
  motivation        jsonb,
  department_fit    public.department,
  existing_experience jsonb,
  resources_runway  jsonb,
  availability      jsonb,
  documents_status  jsonb,
```

```
  medical_next_step text,           -- "Consult a maritime medical provider" etc.
  travel_permissions jsonb,
  expectations      jsonb,
  -- Coach review
  coach_id          uuid REFERENCES public.profiles(id),
  coach_notes       text,
  recommended_pathway text,
  pathway_rationale text,           -- "Why this pathway fits"
  uncertainties     text,           -- "What remains uncertain"
  do_not_spend_yet  text,           -- "What NOT to spend money on yet"
  reviewed_at       timestamptz,
  created_at        timestamptz NOT NULL DEFAULT now(),
  updated_at        timestamptz NOT NULL DEFAULT now()
);

-- Pathway versions (versioned, never mutated once agreed)
CREATE TABLE public.pathway_versions (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  assessment_id     uuid REFERENCES public.assessments(id),
  version           integer NOT NULL DEFAULT 1,
  department        public.department NOT NULL,
  target_role       text NOT NULL,          -- e.g. "Deckhand", "Steward/ess"
  summary           text,
  is_active         boolean NOT NULL DEFAULT true,
  approved_by       uuid REFERENCES public.profiles(id),
  approved_at       timestamptz,
  created_at        timestamptz NOT NULL DEFAULT now(),
  UNIQUE(user_id, version)
);

-- Requirements engine (versioned, sourced, never hardcoded advice)
CREATE TABLE public.requirements (
  id                  uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  code                text NOT NULL,                  -- e.g. "STCW_BASIC_SAFETY"
  title               text NOT NULL,
  description         text,
  classification      public.requirement_classification NOT NULL,
  applicable_flags    text[] DEFAULT '{}',             -- e.g. ['MCA', 'SAMSA']
  applicable_roles    text[] DEFAULT '{}',
  applicable_departments public.department[],
  source_authority    text NOT NULL,                   -- e.g. "IMO STCW
Convention"
  source_url          text,
  source_version      text,                            -- e.g. "2010 Manila
Amendments"
```

```
  last_verified_date  date NOT NULL,
  verified_by         text NOT NULL,                  -- Person/role who verified
  prerequisites       uuid[] DEFAULT '{}',            -- Refs to other
requirements
  notes               text,
  is_active           boolean NOT NULL DEFAULT true,
  created_at          timestamptz NOT NULL DEFAULT now(),
  updated_at          timestamptz NOT NULL DEFAULT now()
);

-- Milestones (steps within a pathway)
CREATE TABLE public.milestones (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  pathway_id      uuid NOT NULL REFERENCES public.pathway_versions(id) ON DELETE
CASCADE,
  requirement_id  uuid REFERENCES public.requirements(id),
  title           text NOT NULL,
  description     text,
  sort_order      integer NOT NULL DEFAULT 0,
  evidence_required text,               -- What evidence must be uploaded
  status          text NOT NULL DEFAULT 'pending'
                  CHECK (status IN ('pending', 'in_progress', 'submitted',
'approved', 'deferred', 'not_applicable')),
  approved_by     uuid REFERENCES public.profiles(id),
  approved_at     timestamptz,
  notes           text,
  created_at      timestamptz NOT NULL DEFAULT now(),
  updated_at      timestamptz NOT NULL DEFAULT now()
);

-- Journey state transitions (audit trail)
CREATE TABLE public.journey_transitions (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  from_state      public.journey_state,
  to_state        public.journey_state NOT NULL,
  reason          text,
  evidence        jsonb,                -- What evidence supported this
  reviewed_by     uuid REFERENCES public.profiles(id),
  created_at      timestamptz NOT NULL DEFAULT now()
);

-- Current journey state on profile
ALTER TABLE public.profiles
  ADD COLUMN journey_state public.journey_state DEFAULT 'exploring';

CREATE TRIGGER set_assessments_updated_at
```

```
  BEFORE UPDATE ON public.assessments
  FOR EACH ROW EXECUTE FUNCTION public.handle_updated_at();

CREATE TRIGGER set_requirements_updated_at
  BEFORE UPDATE ON public.requirements
  FOR EACH ROW EXECUTE FUNCTION public.handle_updated_at();

CREATE TRIGGER set_milestones_updated_at
  BEFORE UPDATE ON public.milestones
  FOR EACH ROW EXECUTE FUNCTION public.handle_updated_at();
```

2.3 Migration 00003 — Learning

```
sql
-- supabase/migrations/00003_learning.sql

CREATE TYPE public.lesson_type AS ENUM ('video', 'text', 'quiz', 'practical',
'live');

CREATE TABLE public.programmes (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title         text NOT NULL,
  slug          text UNIQUE NOT NULL,
  description   text,
  department    public.department,
  duration_weeks integer,
  is_active     boolean NOT NULL DEFAULT true,
  created_at    timestamptz NOT NULL DEFAULT now(),
  updated_at    timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE public.cohorts (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  programme_id  uuid NOT NULL REFERENCES public.programmes(id),
  name          text NOT NULL,               -- e.g. "Deck Jan 2027"
  starts_at     date NOT NULL,
  ends_at       date,
  max_members   integer DEFAULT 12,
  coach_id      uuid REFERENCES public.profiles(id),
  is_active     boolean NOT NULL DEFAULT true,
  created_at    timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE public.cohort_members (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  cohort_id   uuid NOT NULL REFERENCES public.cohorts(id) ON DELETE CASCADE,
  user_id     uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  enrolled_at timestamptz NOT NULL DEFAULT now(),
  status      text NOT NULL DEFAULT 'active'
              CHECK (status IN ('active', 'paused', 'completed', 'withdrawn')),
  UNIQUE(cohort_id, user_id)
);

CREATE TABLE public.lessons (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  programme_id  uuid NOT NULL REFERENCES public.programmes(id) ON DELETE CASCADE,
  title         text NOT NULL,
```

```
  slug          text NOT NULL,
  lesson_type   public.lesson_type NOT NULL,
  content       text,                  -- Markdown/rich text
  video_url     text,                  -- Signed URL or Cloudflare Stream ID
  sort_order    integer NOT NULL DEFAULT 0,
  duration_mins integer,
  is_published  boolean NOT NULL DEFAULT false,
  created_at    timestamptz NOT NULL DEFAULT now(),
  updated_at    timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE public.assignments (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  lesson_id     uuid NOT NULL REFERENCES public.lessons(id) ON DELETE CASCADE,
  title         text NOT NULL,
  instructions  text NOT NULL,
  due_offset_days integer,            -- Days after cohort start
  submission_type text NOT NULL DEFAULT 'text'
                  CHECK (submission_type IN ('text', 'file', 'quiz',
'practical_video')),
  max_score     integer,
  created_at    timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE public.assignment_submissions (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  assignment_id   uuid NOT NULL REFERENCES public.assignments(id),
  user_id         uuid NOT NULL REFERENCES public.profiles(id),
  cohort_id       uuid NOT NULL REFERENCES public.cohorts(id),
  content         text,
  file_path       text,                 -- Storage path
  submitted_at    timestamptz NOT NULL DEFAULT now(),
  -- Coach review
  score           integer,
  feedback        text,
  reviewed_by     uuid REFERENCES public.profiles(id),
  reviewed_at     timestamptz,
  status          text NOT NULL DEFAULT 'submitted'
                  CHECK (status IN ('draft', 'submitted', 'reviewed',
'resubmit_requested'))
);

CREATE TABLE public.lesson_progress (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  lesson_id   uuid NOT NULL REFERENCES public.lessons(id) ON DELETE CASCADE,
```

```
  completed   boolean NOT NULL DEFAULT false,
  completed_at timestamptz,
  created_at  timestamptz NOT NULL DEFAULT now(),
  UNIQUE(user_id, lesson_id)
);

-- Live sessions (links to external meeting provider)
CREATE TABLE public.live_sessions (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  cohort_id       uuid NOT NULL REFERENCES public.cohorts(id),
  title           text NOT NULL,
  description     text,
  scheduled_at    timestamptz NOT NULL,
  duration_mins   integer NOT NULL DEFAULT 60,
  meeting_url     text,                    -- Zoom/external join link
  meeting_id      text,                    -- External provider ID
  recording_url   text,                    -- Cloudflare Stream signed URL
(feature-flagged)
  recording_consent_required boolean DEFAULT true,
  created_at      timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE public.session_attendance (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id  uuid NOT NULL REFERENCES public.live_sessions(id) ON DELETE CASCADE,
  user_id     uuid NOT NULL REFERENCES public.profiles(id),
  joined_at   timestamptz,
  left_at     timestamptz,
  recording_consent boolean DEFAULT false,
  UNIQUE(session_id, user_id)
);

CREATE TABLE public.session_questions (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id  uuid NOT NULL REFERENCES public.live_sessions(id) ON DELETE CASCADE,
  user_id     uuid NOT NULL REFERENCES public.profiles(id),
  question    text NOT NULL,
  answered    boolean DEFAULT false,
  answer      text,
  created_at  timestamptz NOT NULL DEFAULT now()
);

CREATE TRIGGER set_programmes_updated_at
  BEFORE UPDATE ON public.programmes
  FOR EACH ROW EXECUTE FUNCTION public.handle_updated_at();

CREATE TRIGGER set_lessons_updated_at
```

```
BEFORE UPDATE ON public.lessons
FOR EACH ROW EXECUTE FUNCTION public.handle_updated_at();
```

2.4 Migration 00004 — Evidence

```
sql
-- supabase/migrations/00004_evidence.sql

CREATE TYPE public.document_category AS ENUM (
  'identity',           -- Passport, visa, CoC
  'medical',            -- ENG1, PEME, equivalent
  'training',           -- STCW, safety certs
  'qualification',      -- Academic, professional
  'reference',          -- Employment references
  'travel',             -- Visa, work permits
  'other'
);

CREATE TYPE public.verification_status AS ENUM (
  'claimed',            -- Member says they have it
  'uploaded',           -- File uploaded
  'reviewed',           -- Coach has looked at it
  'verified'            -- Confirmed with issuing body or employer
);

CREATE TABLE public.documents (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  category          public.document_category NOT NULL,
  title             text NOT NULL,
  description       text,
  -- We store STATUS and EXPIRY, not raw sensitive content metadata
  verification_status public.verification_status NOT NULL DEFAULT 'claimed',
  file_path         text,                       -- Supabase Storage path (private
bucket)
  file_name         text,
  file_size_bytes   integer,
  mime_type         text,
  issued_date       date,
  expiry_date       date,
  issuing_authority text,
  document_number_hash text,                    -- Hashed, not plaintext
  -- Review
  reviewed_by       uuid REFERENCES public.profiles(id),
  reviewed_at       timestamptz,
  review_notes      text,
  -- Flags
  expiry_warning_sent boolean DEFAULT false,
  is_sensitive      boolean NOT NULL DEFAULT false,  -- Never sent to AI
```

```
  created_at        timestamptz NOT NULL DEFAULT now(),
  updated_at        timestamptz NOT NULL DEFAULT now()
);

-- Credential verification records
CREATE TABLE public.credential_verifications (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id     uuid NOT NULL REFERENCES public.documents(id) ON DELETE CASCADE,
  verified_by     uuid NOT NULL REFERENCES public.profiles(id),
  method          text NOT NULL CHECK (method IN ('visual_check',
'issuer_confirmed', 'database_lookup', 'employer_confirmed')),
  result          text NOT NULL CHECK (result IN ('confirmed', 'unable_to_verify',
'discrepancy', 'expired')),
  notes           text,
  verified_at     timestamptz NOT NULL DEFAULT now()
);

-- General audit events
CREATE TABLE public.audit_events (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid,                           -- Who was affected
  actor_id    uuid,                           -- Who performed the action
  event_type  text NOT NULL,                  -- e.g. 'document.uploaded',
'state.transition'
  resource    text,                           -- e.g. 'documents', 'cv_versions'
  resource_id uuid,
  details     jsonb DEFAULT '{}',
  ip_address  inet,
  created_at  timestamptz NOT NULL DEFAULT now()
);

-- Data export requests (POPIA right of access)
CREATE TABLE public.data_export_requests (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid NOT NULL REFERENCES public.profiles(id),
  status      text NOT NULL DEFAULT 'pending'
              CHECK (status IN ('pending', 'processing', 'ready', 'downloaded',
'expired')),
  file_path   text,
  requested_at timestamptz NOT NULL DEFAULT now(),
  completed_at timestamptz,
  expires_at  timestamptz
);

-- Data deletion requests (POPIA right to deletion)
CREATE TABLE public.data_deletion_requests (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  user_id     uuid NOT NULL REFERENCES public.profiles(id),
  status      text NOT NULL DEFAULT 'pending'
              CHECK (status IN ('pending', 'approved', 'processing', 'completed',
'denied')),
  reason      text,
  approved_by uuid REFERENCES public.profiles(id),
  requested_at timestamptz NOT NULL DEFAULT now(),
  completed_at timestamptz
);

CREATE TRIGGER set_documents_updated_at
  BEFORE UPDATE ON public.documents
  FOR EACH ROW EXECUTE FUNCTION public.handle_updated_at();
```

2.5 Migration 00005 — Career

```
sql
-- supabase/migrations/00005_career.sql

CREATE TYPE public.cv_data_label AS ENUM (
  'claimed',
  'uploaded',
  'reviewed',
  'verified'
);

CREATE TABLE public.cv_versions (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  version         integer NOT NULL DEFAULT 1,
  -- Structured sections (JSON blocks, each field has a label)
  personal_info   jsonb DEFAULT '{}',       -- name, nationality, etc.
  objective       jsonb DEFAULT '{}',       -- {text, label: cv_data_label}
  experience      jsonb DEFAULT '[]',       -- [{title, employer, dates,
description, label}]
  qualifications  jsonb DEFAULT '[]',       -- [{name, issuer, date, label}]
  skills          jsonb DEFAULT '[]',
  references      jsonb DEFAULT '[]',       -- [{name, relation, contact_shared,
label}]
  additional      jsonb DEFAULT '{}',
  -- Metadata
  coach_comments  text,
  is_current      boolean NOT NULL DEFAULT true,
  pdf_path        text,                     -- Generated PDF storage path
  created_at      timestamptz NOT NULL DEFAULT now(),
  updated_at      timestamptz NOT NULL DEFAULT now(),
  UNIQUE(user_id, version)
);

-- Employers (approved partners only)
CREATE TABLE public.employers (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name            text NOT NULL,
  contact_name    text,
  contact_email   text,
  website         text,
  is_verified     boolean NOT NULL DEFAULT false,
  verification_notes text,
  created_at      timestamptz NOT NULL DEFAULT now(),
  updated_at      timestamptz NOT NULL DEFAULT now()
```

```
);

-- Opportunities (approved partner listings + outbound links)
CREATE TABLE public.opportunities (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  employer_id     uuid REFERENCES public.employers(id),
  title           text NOT NULL,
  department      public.department,
  description     text,
  location        text,
  vessel_type     text,
  contract_type   text CHECK (contract_type IN ('seasonal', 'permanent', 'daywork',
'rotational')),
  external_url    text,                   -- Outbound link to original listing
  source          text NOT NULL DEFAULT 'partner'
                  CHECK (source IN ('partner', 'direct_submission')),
  is_active       boolean NOT NULL DEFAULT true,
  posted_at       timestamptz NOT NULL DEFAULT now(),
  expires_at      timestamptz,
  created_at      timestamptz NOT NULL DEFAULT now()
);

-- Applications
CREATE TABLE public.applications (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         uuid NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  opportunity_id  uuid REFERENCES public.opportunities(id),
  -- Can also track external applications
  external_title  text,
  external_employer text,
  external_url    text,
  status          text NOT NULL DEFAULT 'saved'
                  CHECK (status IN (
                    'saved', 'preparing', 'submitted', 'acknowledged',
                    'interview_scheduled', 'interviewed', 'offered',
                    'accepted', 'declined', 'rejected', 'withdrawn'
                  )),
  cv_version_id   uuid REFERENCES public.cv_versions(id),
  cover_note      text,
  submitted_at    timestamptz,
  notes           text,
  created_at      timestamptz NOT NULL DEFAULT now(),
  updated_at      timestamptz NOT NULL DEFAULT now()
);

-- Interviews
```

```
CREATE TABLE public.interviews (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  application_id  uuid NOT NULL REFERENCES public.applications(id) ON DELETE
CASCADE,
  scheduled_at    timestamptz,
  format          text CHECK (format IN ('in_person', 'video', 'phone')),
  location        text,
  interviewer     text,
  notes           text,
  outcome         text CHECK (outcome IN ('pending', 'positive', 'negative',
'no_feedback')),
  coach_debrief   text,
  created_at      timestamptz NOT NULL DEFAULT now()
);

CREATE TRIGGER set_cv_versions_updated_at
  BEFORE UPDATE ON public.cv_versions
  FOR EACH ROW EXECUTE FUNCTION public.handle_updated_at();

CREATE TRIGGER set_applications_updated_at
  BEFORE UPDATE ON public.applications
  FOR EACH ROW EXECUTE FUNCTION public.handle_updated_at();
```

2.6 Migration 00006 — Outcomes

```
sql
-- supabase/migrations/00006_outcomes.sql

CREATE TYPE public.employment_type AS ENUM ('seasonal', 'permanent', 'daywork',
'rotational');
CREATE TYPE public.outcome_evidence_type AS ENUM (
  'redacted_contract',
  'employer_confirmation',
  'start_confirmation',
  'member_self_report'
);

CREATE TABLE public.offers (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         uuid NOT NULL REFERENCES public.profiles(id),
  application_id  uuid REFERENCES public.applications(id),
  employer_id     uuid REFERENCES public.employers(id),
  role_title      text NOT NULL,
  vessel_name     text,
  employment_type public.employment_type,
  start_date      date,
  salary_range    text,                    -- Redacted range, not exact
  status          text NOT NULL DEFAULT 'pending_review'
                  CHECK (status IN ('pending_review', 'coach_reviewed', 'accepted',
'declined', 'rescinded')),
  coach_review    text,                    -- Coach commentary on the offer
  reviewed_by     uuid REFERENCES public.profiles(id),
  reviewed_at     timestamptz,
  created_at      timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE public.verified_starts (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         uuid NOT NULL REFERENCES public.profiles(id),
  offer_id        uuid REFERENCES public.offers(id),
  evidence_type   public.outcome_evidence_type NOT NULL,
  evidence_notes  text,
  start_date      date NOT NULL,
  verified_by     uuid REFERENCES public.profiles(id),
  verified_at     timestamptz NOT NULL DEFAULT now(),
  -- Consistent with agreed pathway?
  consistent_with_pathway boolean,
  pathway_notes   text,
  created_at      timestamptz NOT NULL DEFAULT now()
```

```
);

CREATE TABLE public.retention_checks (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         uuid NOT NULL REFERENCES public.profiles(id),
  verified_start_id uuid REFERENCES public.verified_starts(id),
  check_type      text NOT NULL CHECK (check_type IN (
    'week_1', 'week_4', 'month_3', 'season_end', 'ad_hoc'
  )),
  status          text NOT NULL DEFAULT 'pending'
                  CHECK (status IN ('pending', 'completed', 'unreachable',
'departed')),
  wellbeing_score integer CHECK (wellbeing_score BETWEEN 1 AND 5),
  notes           text,
  coach_id        uuid REFERENCES public.profiles(id),
  checked_at      timestamptz,
  created_at      timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE public.withdrawal_reasons (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid NOT NULL REFERENCES public.profiles(id),
  stage       public.journey_state NOT NULL,       -- State at withdrawal
  reason      text NOT NULL,
  details     text,
  recorded_by uuid REFERENCES public.profiles(id),
  created_at  timestamptz NOT NULL DEFAULT now()
);
```

2.7 Migration 00007 — RLS Policies

```
sql
-- supabase/migrations/00007_rls_policies.sql

-- Enable RLS on all tables
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.organisations ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.coach_assignments ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.consent_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.marketing_suppressions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.access_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.assessments ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.pathway_versions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.requirements ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.milestones ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.journey_transitions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.programmes ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.cohorts ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.cohort_members ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.lessons ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.assignments ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.assignment_submissions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.lesson_progress ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.live_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.session_attendance ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.session_questions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.credential_verifications ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.audit_events ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.data_export_requests ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.data_deletion_requests ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.cv_versions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.employers ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.opportunities ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.applications ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.interviews ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.offers ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.verified_starts ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.retention_checks ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.withdrawal_reasons ENABLE ROW LEVEL SECURITY;

-- ============================================================
-- HELPER: Check if current user is coach for a given member
-- ============================================================
CREATE OR REPLACE FUNCTION public.is_coach_of(member_uuid uuid)
```

```
RETURNS boolean AS $$
  SELECT EXISTS (
    SELECT 1 FROM public.coach_assignments
    WHERE coach_id = auth.uid()
      AND member_id = member_uuid
      AND ended_at IS NULL
  );
$$ LANGUAGE sql SECURITY DEFINER STABLE;

CREATE OR REPLACE FUNCTION public.current_user_role()
RETURNS public.user_role AS $$
  SELECT role FROM public.profiles WHERE id = auth.uid();
$$ LANGUAGE sql SECURITY DEFINER STABLE;

-- ============================================================
-- PROFILES
-- ============================================================
-- Members see own profile
CREATE POLICY profiles_select_own ON public.profiles
  FOR SELECT USING (id = auth.uid());

-- Coaches see assigned members
CREATE POLICY profiles_select_coach ON public.profiles
  FOR SELECT USING (
    public.current_user_role() = 'coach'
    AND (id = auth.uid() OR public.is_coach_of(id))
  );

-- Admins see all
CREATE POLICY profiles_select_admin ON public.profiles
  FOR SELECT USING (public.current_user_role() = 'admin');

-- Members update own
CREATE POLICY profiles_update_own ON public.profiles
  FOR UPDATE USING (id = auth.uid())
  WITH CHECK (id = auth.uid());

-- Coaches can update assigned members (limited fields via app layer)
CREATE POLICY profiles_update_coach ON public.profiles
  FOR UPDATE USING (
    public.current_user_role() = 'coach' AND public.is_coach_of(id)
  );

-- ============================================================
-- ASSESSMENTS
-- ============================================================
CREATE POLICY assessments_select_own ON public.assessments
```

```
  FOR SELECT USING (user_id = auth.uid());

CREATE POLICY assessments_select_coach ON public.assessments
  FOR SELECT USING (
    public.current_user_role() IN ('coach', 'admin')
    AND public.is_coach_of(user_id)
  );

CREATE POLICY assessments_insert_own ON public.assessments
  FOR INSERT WITH CHECK (user_id = auth.uid());

CREATE POLICY assessments_update_own ON public.assessments
  FOR UPDATE USING (user_id = auth.uid() AND status = 'in_progress');

CREATE POLICY assessments_update_coach ON public.assessments
  FOR UPDATE USING (
    public.current_user_role() = 'coach' AND public.is_coach_of(user_id)
  );

-- ============================================================
-- DOCUMENTS — Members see own; coaches see assigned; NEVER searchable vault
-- ============================================================
CREATE POLICY documents_select_own ON public.documents
  FOR SELECT USING (user_id = auth.uid());

CREATE POLICY documents_select_coach ON public.documents
  FOR SELECT USING (
    public.current_user_role() IN ('coach', 'admin')
    AND public.is_coach_of(user_id)
  );

CREATE POLICY documents_insert_own ON public.documents
  FOR INSERT WITH CHECK (user_id = auth.uid());

CREATE POLICY documents_update_own ON public.documents
  FOR UPDATE USING (user_id = auth.uid());

CREATE POLICY documents_update_coach ON public.documents
  FOR UPDATE USING (
    public.current_user_role() = 'coach' AND public.is_coach_of(user_id)
  );

-- ============================================================
-- CV VERSIONS
-- ============================================================
CREATE POLICY cv_select_own ON public.cv_versions
  FOR SELECT USING (user_id = auth.uid());
```

```
CREATE POLICY cv_select_coach ON public.cv_versions
  FOR SELECT USING (
    public.current_user_role() IN ('coach', 'admin')
    AND public.is_coach_of(user_id)
  );

CREATE POLICY cv_insert_own ON public.cv_versions
  FOR INSERT WITH CHECK (user_id = auth.uid());

CREATE POLICY cv_update_own ON public.cv_versions
  FOR UPDATE USING (user_id = auth.uid());

CREATE POLICY cv_update_coach ON public.cv_versions
  FOR UPDATE USING (
    public.current_user_role() = 'coach' AND public.is_coach_of(user_id)
  );

-- ============================================================
-- REQUIREMENTS — Public read (reference data)
-- ============================================================
CREATE POLICY requirements_select_all ON public.requirements
  FOR SELECT USING (true);

-- Only admin/coach can insert/update
CREATE POLICY requirements_manage ON public.requirements
  FOR ALL USING (public.current_user_role() IN ('coach', 'admin'));

-- ============================================================
-- PATHWAY VERSIONS
-- ============================================================
CREATE POLICY pathway_select_own ON public.pathway_versions
  FOR SELECT USING (user_id = auth.uid());

CREATE POLICY pathway_select_coach ON public.pathway_versions
  FOR SELECT USING (
    public.current_user_role() IN ('coach', 'admin')
    AND public.is_coach_of(user_id)
  );

-- ============================================================
-- MILESTONES
-- ============================================================
CREATE POLICY milestones_select_own ON public.milestones
  FOR SELECT USING (
    EXISTS (
      SELECT 1 FROM public.pathway_versions pv
      WHERE pv.id = pathway_id AND pv.user_id = auth.uid()
```

```
    )
  );

CREATE POLICY milestones_select_coach ON public.milestones
  FOR SELECT USING (
    public.current_user_role() IN ('coach', 'admin')
    AND EXISTS (
      SELECT 1 FROM public.pathway_versions pv
      WHERE pv.id = pathway_id AND public.is_coach_of(pv.user_id)
    )
  );

-- ============================================================
-- OPPORTUNITIES — All authenticated can read active listings
-- ============================================================
CREATE POLICY opportunities_select ON public.opportunities
  FOR SELECT USING (is_active = true);

CREATE POLICY opportunities_manage ON public.opportunities
  FOR ALL USING (public.current_user_role() IN ('coach', 'admin'));

-- ============================================================
-- APPLICATIONS — Members see own; coaches see assigned
-- ============================================================
CREATE POLICY applications_select_own ON public.applications
  FOR SELECT USING (user_id = auth.uid());

CREATE POLICY applications_select_coach ON public.applications
  FOR SELECT USING (
    public.current_user_role() IN ('coach', 'admin')
    AND public.is_coach_of(user_id)
  );

CREATE POLICY applications_insert_own ON public.applications
  FOR INSERT WITH CHECK (user_id = auth.uid());

CREATE POLICY applications_update_own ON public.applications
  FOR UPDATE USING (user_id = auth.uid());

-- ============================================================
-- CONSENT RECORDS — Own only
-- ============================================================
CREATE POLICY consent_select_own ON public.consent_records
  FOR SELECT USING (user_id = auth.uid());

CREATE POLICY consent_insert_own ON public.consent_records
  FOR INSERT WITH CHECK (user_id = auth.uid());
```

```
-- Cannot delete/update consent — only new records

-- ============================================================
-- LEARNING (lessons, programmes, cohorts accessible to enrolled members)
-- ============================================================
CREATE POLICY programmes_select ON public.programmes
  FOR SELECT USING (is_active = true);

CREATE POLICY lessons_select_enrolled ON public.lessons
  FOR SELECT USING (
    is_published = true
    AND EXISTS (
      SELECT 1 FROM public.cohort_members cm
      JOIN public.cohorts c ON c.id = cm.cohort_id
      WHERE cm.user_id = auth.uid()
        AND c.programme_id = lessons.programme_id
        AND cm.status = 'active'
    )
  );

CREATE POLICY lessons_select_coach ON public.lessons
  FOR SELECT USING (public.current_user_role() IN ('coach', 'admin'));

CREATE POLICY submissions_select_own ON public.assignment_submissions
  FOR SELECT USING (user_id = auth.uid());

CREATE POLICY submissions_select_coach ON public.assignment_submissions
  FOR SELECT USING (
    public.current_user_role() IN ('coach', 'admin')
    AND public.is_coach_of(user_id)
  );

CREATE POLICY submissions_insert_own ON public.assignment_submissions
  FOR INSERT WITH CHECK (user_id = auth.uid());

-- ============================================================
-- OUTCOMES — Coaches and admin only; members see own offers
-- ============================================================
CREATE POLICY offers_select_own ON public.offers
  FOR SELECT USING (user_id = auth.uid());

CREATE POLICY offers_select_coach ON public.offers
  FOR SELECT USING (public.current_user_role() IN ('coach', 'admin'));

CREATE POLICY verified_starts_coach ON public.verified_starts
  FOR ALL USING (public.current_user_role() IN ('coach', 'admin'));
```

```
CREATE POLICY retention_checks_coach ON public.retention_checks
  FOR ALL USING (public.current_user_role() IN ('coach', 'admin'));

-- ============================================================
-- AUDIT EVENTS — Admin only read; system insert
-- ============================================================
CREATE POLICY audit_select_admin ON public.audit_events
  FOR SELECT USING (public.current_user_role() = 'admin');
```

2.8 Migration 00008 — Functions (Journey State Guards)

```
sql
-- supabase/migrations/00008_functions.sql

-- Transition guard: validates allowed state transitions
CREATE OR REPLACE FUNCTION public.transition_journey_state(
  p_user_id uuid,
  p_new_state public.journey_state,
  p_reason text DEFAULT NULL,
  p_evidence jsonb DEFAULT '{}'
)
RETURNS void AS $$
DECLARE
  v_current_state public.journey_state;
  v_allowed boolean := false;
BEGIN
  SELECT journey_state INTO v_current_state
  FROM public.profiles WHERE id = p_user_id;

  -- Define allowed transitions
  v_allowed := CASE
    -- Forward progression
    WHEN v_current_state = 'exploring'              AND p_new_state IN
('pathway_agreed', 'paused', 'alternative_route') THEN true
    WHEN v_current_state = 'pathway_agreed'         AND p_new_state IN
('preparing', 'paused', 'needs_review', 'alternative_route') THEN true
    WHEN v_current_state = 'preparing'              AND p_new_state IN
('training_complete', 'paused', 'needs_review') THEN true
    WHEN v_current_state = 'training_complete'      AND p_new_state IN
('profile_ready', 'needs_review') THEN true
    WHEN v_current_state = 'profile_ready'          AND p_new_state IN
('search_active', 'needs_review') THEN true
    WHEN v_current_state = 'search_active'          AND p_new_state IN
('interviewing', 'paused', 'needs_review') THEN true
    WHEN v_current_state = 'interviewing'           AND p_new_state IN ('offer',
'search_active', 'needs_review') THEN true
    WHEN v_current_state = 'offer'                  AND p_new_state IN
('employment_started', 'search_active', 'needs_review') THEN true
    WHEN v_current_state = 'employment_started'     AND p_new_state IN
('first_season_supported', 'needs_review') THEN true
    -- Recovery from non-linear states
    WHEN v_current_state = 'paused'                 AND p_new_state IN
('exploring', 'pathway_agreed', 'preparing', 'search_active') THEN true
    WHEN v_current_state = 'needs_review'           AND p_new_state IS NOT NULL
THEN true  -- Coach can move to any state
```

```
    WHEN v_current_state = 'alternative_route'      AND p_new_state IN
('exploring', 'paused') THEN true
    ELSE false
  END;

  IF NOT v_allowed THEN
    RAISE EXCEPTION 'Invalid transition from % to %', v_current_state, p_new_state;
  END IF;

  -- Record transition
  INSERT INTO public.journey_transitions (user_id, from_state, to_state, reason,
evidence, reviewed_by)
  VALUES (p_user_id, v_current_state, p_new_state, p_reason, p_evidence,
auth.uid());

  -- Update profile
  UPDATE public.profiles SET journey_state = p_new_state WHERE id = p_user_id;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Audit trigger: logs all changes to sensitive tables
CREATE OR REPLACE FUNCTION public.audit_change()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.audit_events (user_id, actor_id, event_type, resource,
resource_id, details)
  VALUES (
    COALESCE(NEW.user_id, OLD.user_id),
    auth.uid(),
    TG_OP,
    TG_TABLE_NAME,
    COALESCE(NEW.id, OLD.id),
    jsonb_build_object('old', to_jsonb(OLD), 'new', to_jsonb(NEW))
  );
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Apply audit triggers to key tables
CREATE TRIGGER audit_documents AFTER INSERT OR UPDATE OR DELETE ON public.documents
  FOR EACH ROW EXECUTE FUNCTION public.audit_change();

CREATE TRIGGER audit_journey_transitions AFTER INSERT ON public.journey_transitions
  FOR EACH ROW EXECUTE FUNCTION public.audit_change();

CREATE TRIGGER audit_cv_versions AFTER INSERT OR UPDATE ON public.cv_versions
  FOR EACH ROW EXECUTE FUNCTION public.audit_change();
```

```
CREATE TRIGGER audit_offers AFTER INSERT OR UPDATE ON public.offers
  FOR EACH ROW EXECUTE FUNCTION public.audit_change();

CREATE TRIGGER audit_verified_starts AFTER INSERT ON public.verified_starts
  FOR EACH ROW EXECUTE FUNCTION public.audit_change();
```

2.9 Migration 00009 — Feature Flags

```
sql
-- supabase/migrations/00009_feature_flags.sql

CREATE TABLE public.feature_flags (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  key         text UNIQUE NOT NULL,
  enabled     boolean NOT NULL DEFAULT false,
  description text,
  created_at  timestamptz NOT NULL DEFAULT now(),
  updated_at  timestamptz NOT NULL DEFAULT now()
);

ALTER TABLE public.feature_flags ENABLE ROW LEVEL SECURITY;

-- Everyone can read flags
CREATE POLICY flags_select ON public.feature_flags
  FOR SELECT USING (true);

-- Only admin can modify
CREATE POLICY flags_manage ON public.feature_flags
  FOR ALL USING (public.current_user_role() = 'admin');

-- Seed flags
INSERT INTO public.feature_flags (key, enabled, description) VALUES
  ('payments_enabled',         false, 'Payment checkout — disabled pending legal
review (Employment Services Act s.15)'),
  ('ai_cv_assist',             false, 'AI-powered CV wording suggestions — server-
side only'),
  ('ai_lesson_search',         false, 'Semantic lesson search'),
  ('ai_summarisation',         false, 'Session summarisation'),
  ('cloudflare_stream',        false, 'Cloudflare Stream for replays — signed
playback'),
  ('employer_portal',          false, 'Employer self-service portal'),
  ('ghl_sync',                 false, 'GoHighLevel CRM downstream sync'),
  ('map_directory',            false, 'MapLibre map directory'),
  ('engineering_department',   false, 'Engineering department pathways — requires
specialist coach'),
  ('galley_department',        false, 'Galley department pathways — requires
specialist coach');
```

3. Route Map

Route	Role	Purpose
`/`	Public	Landing page
`/login`	Public	Sign in
`/register`	Public	Sign up
`/forgot-password`	Public	Password reset
`/callback`	Public	OAuth callback
`/today`	Member	Dashboard: next action, coaching, blockers
`/intake`	Member	"Find my route" structured assessment wizard
`/intake/result`	Member	Assessment outcome + rationale
`/pathway`	Member	Milestone timeline + journey state
`/pathway/[milestoneId]`	Member	Milestone detail + evidence submission
`/learn`	Member	Programme & lesson list (cohort-gated)
`/learn/[lessonId]`	Member	Lesson content
`/learn/assignments`	Member	Assignment list
`/learn/assignments/[assignmentId]`	Member	Submit / view assignment
`/live`	Member	Session calendar
`/live/[sessionId]`	Member	Session detail + join link + questions
`/documents`	Member	Document upload and status
`/documents/[documentId]`	Member	Document detail / verification status
`/cv`	Member	CV Studio structured editor
`/cv/preview`	Member	PDF preview
`/cv/history`	Member	Version history + coach comments
`/explore`	Member	Destination resources
`/explore/map`	Member	MapLibre directory (feature-flagged)
`/opportunities`	Member	Approved partner listings
`/opportunities/[opportunityId]`	Member	Listing detail + outbound link

Route	Role	Purpose
`/applications`	Member	Application tracker
`/applications/[applicationId]`	Member	Application detail / follow-up
`/support`	Member	Coach requests, feedback, challenge request
`/settings`	Member	Profile, preferences
`/settings/consent`	Member	POPIA consent management
`/console`	Coach	Intervention-first dashboard
`/console/members`	Coach	Assigned members list
`/console/members/[memberId]`	Coach	Deep member view (pathway, docs, CV, applications)
`/console/reviews`	Coach	Pending evidence reviews queue
`/console/cohorts`	Coach	Cohort management
`/console/cohorts/[cohortId]`	Coach	Cohort detail + attendance + progress
`/console/outcomes`	Coach	Outcome recording dashboard
`/console/settings`	Coach	Coach profile and preferences

4. Build Order (Vertical Slices)

#	Slice	Tables Involved	Key Deliverables
1	Auth + Roles	profiles, organisations, coach_assignments, consent_records	Supabase Auth, auto-profile creation, role routing middleware, login/register UI
2	Intake Assessment	assessments	Multi-step wizard, coach review view, result page with rationale
3	Pathway + Journey States	pathway_versions, requirements, milestones, journey_transitions	Timeline UI, state machine with guards, transition audit trail
4	Documents + Evidence Review	documents, credential_verifications, audit_events	Upload to private bucket, signed URLs, coach review workflow, expiry tracking
5	CV Studio	cv_versions	Structured editor, data labels, version history, PDF export, coach comments
6	Learning / Cohorts	programmes, cohorts, cohort_members, lessons, assignments, etc.	Lesson viewer, assignment submission, quiz, progress tracking, cohort enrolment
7	Live Room	live_sessions, session_attendance, session_questions	Calendar, Zoom join links, recording consent, question queue
8	Applications Tracker	employers, opportunities, applications, interviews	Opportunity browser, application status board, interview logging
9	Coach Console	(cross-cutting reads)	Intervention queue, member case view, readiness matrix, diagnostic flags
10	Outcome Recording	offers, verified_starts, retention_checks, withdrawal_reasons	Offer review, start verification, retention check scheduling, withdrawal capture

5. Feature Flags Summary

Flag	Default	Reason
`payments_enabled`	`false`	**Legal review required —** Employment Services Act s.15 (SA)
`ai_cv_assist`	`false`	Requires API key, safety review, no fabrication guardrails validated
`ai_lesson_search`	`false`	Requires embedding pipeline setup
`ai_summarisation`	`false`	Requires recording pipeline + consent flow
`cloudflare_stream`	`false`	Requires Cloudflare account + signed token setup
`employer_portal`	`false`	MVP uses admin-entered opportunities only
`ghl_sync`	`false`	Requires GoHighLevel API setup + consent checking
`map_directory`	`false`	Requires licensed tile provider + curated location data with provenance
`engineering_department`	`false`	Specialist coach not yet available
`galley_department`	`false`	Specialist coach not yet available

6. Open Decisions Requiring a Human

> IMPORTANT
>
> These questions **cannot be resolved by engineering**. Each needs a named decision-maker (business owner, legal advisor, or domain expert).

Legal / Regulatory

#	Question	Decision-Maker
L1	**Employment Services Act s.15**: What is the legal position on charging members? Can we charge for coaching (not placement)? What documentation do we need?	Legal advisor
L2	**POPIA compliance review**: Do we need a registered Information Officer? What is the data retention period for member records after completion/withdrawal?	Legal advisor
L3	**ENG1 medical equivalents**: Which non-ENG1 medicals are currently accepted by MCA, SAMSA, and major employers? This must be externally verified, not assumed.	Maritime medical advisor
L4	**STCW requirement versions**: Are we tracking STCW 2010 Manila Amendments or have there been updates affecting entry-level deck/interior?	SAMSA / MCA liaison
L5	**Recording consent (SA law)**: Does SA one-party or all-party consent apply to group coaching recordings? Different from POPIA consent?	Legal advisor

Business / Product

#	Question	Decision-Maker
B1	**Pilot pricing model**: Is the first cohort fully free, sponsored, or deposit-refundable? This affects whether the payment module is needed at all for launch.	Chad (founder)
B2	**Coach capacity**: How many members per coach? This affects cohort size, assignment review SLAs, and coach console alert thresholds.	Chad (founder)
B3	**Cohort schedule**: What are the target dates for the first Deck and Interior cohorts? Affects seed data and calendar UI.	Chad (founder)
B4	**External meeting provider**: Confirm Zoom Meeting SDK or alternative (Google Meet, Teams). Affects integration code.	Chad (founder)
B5	**Tile provider for MapLibre**: Mapbox, Maptiler, Protomaps, or self-hosted tiles? Licence and cost implications.	Chad (founder)
B6	**GoHighLevel scope**: Which fields sync to GHL? Only consented marketing contacts, or also journey state?	Chad (founder)
B7	**Outcome definition precision**: Is "daywork" counted as employment outcome or tracked separately? The schema supports both but reporting needs a decision.	Chad (founder)
B8	**Domain and hosting**: Confirm domain name, Vercel vs self-hosted, Supabase cloud vs self-hosted.	Chad (founder)
B9	**Employer verification process**: Who verifies employers, and what evidence is required? This defines the `employers.is_verified` workflow.	Chad (founder)

Technical

#	Question	Decision-Maker
T1	**Supabase project**: Create a new Supabase project now, or develop against local Supabase CLI first?	Chad (founder)
T2	**AI provider for CV assist**: Gemini, OpenAI, or Anthropic? Affects API integration and cost. Server-side only, no PII sent.	Chad (founder)
T3	**PDF generation**: Server-side (Puppeteer/Playwright) or client-side (react-pdf)? Affects deployment (Vercel serverless limits).	Engineering (with Chad)

7. Readiness Matrix Dimensions

The system will **never** reduce employability to a single percentage. Instead, it tracks seven dimensions:

Dimension	Measured By	Source of Truth
Training	Milestone completion against pathway requirements	milestones + documents
Medical	Valid medical certificate uploaded + reviewed (not auto-rejected)	documents
Profile	CV version with coach review, all fields labelled ≥ "reviewed"	cv_versions
Practical Skills	Assignment scores, practical submissions reviewed	assignment_submissions
Travel / Work Permissions	Passport validity, visa status, right-to-work documents	documents
Search Preparation	Applications submitted, interview prep completed	applications
Employment Outcome	Offers, verified starts, retention checks	outcomes tables

Each dimension shows: **Not started · In progress · Submitted · Coach approved · Verified** — never a calculated score.

8. Security Principles (Summary)

1. **RLS everywhere** — no "hidden UI" security. Every table has policies. Tests validate access from each role.
2. **Private Storage bucket** — all document uploads go to a private bucket. Short-lived signed URLs generated server-side.
3. **No PII to AI** — document status and expiry only, never raw identity or medical files.
4. **Consent separation** — marketing consent ≠ programme participation. Each channel (email, SMS, WhatsApp) tracked independently.
5. **Audit trail** — every state transition, document review, and sensitive change logged to `audit_events`.

6. **Account ≠ Readiness —** payment status (when enabled) stored separately. A failed payment never alters qualifications or erases history.
7. **No employer searchable vault —** employers see only explicitly shared candidate info, never a browsable database.
8. **Hashed document numbers —** document identifiers stored as hashes, not plaintext.

Next Step

With this plan reviewed and any open decisions answered, **Slice 1 (Auth + Roles)** will deliver:

- Next.js project scaffolding (App Router, TypeScript, Tailwind)
- Supabase client setup (browser, server, admin)
- Migration 00001 (Identity) + 00009 (Feature Flags) applied
- RLS policies for profiles, consent, coach assignments
- Auth UI: login, register, forgot-password, OAuth callback
- Middleware: role-based routing (member → `/today`, coach → `/console`)
- Auto-profile creation trigger
- `FeatureGate` component
- Member and Coach layout shells with navigation

Ready to build on your confirmation.